

Java 技术管理规范

一、强制

1. 使用 4 个空格作为缩进单位，不允许使用制表符 (Tab) 。
2. 一行代码最大长度为 120 个字符，超长应换行，换行位置应在操作符之前、逗号之后等合适处。
3. 在关键字与括号之间、操作符两侧、方法参数列表中参数之间添加空格。
4. 在方法之间、类的成员变量与方法之间、逻辑块之间添加空行，增强代码可读性。
5. 变量名具有明确语义，反映用途和含义，遵循驼峰命名法（首字母小写，后续单词首字母大写）。
6. 避免使用无意义变量名（如 a、b、c），除非极少数简单场景且不会引起混淆。
7. 类名和接口名使用骆驼命名法（每个单词首字母大写），类名具名词特性，接口名常以 “able”、“ible”、“er” 等结尾。
8. 避免使用模糊或无意义的类名和接口名（如 Class1、MyClass、ObjectInterface）。
9. 方法名使用驼峰命名法，以动词或动词短语开头，准确描述功能和行为。
10. 避免使用模糊或无意义的方法名（如 method1、doSomething），除非特定情况下无法找到合适命名。
11. 类定义顺序：类声明 → 成员变量声明 → 构造方法 → 方法和嵌套类（如有）。
12. 方法参数列表简洁，避免过多参数。参数过多时，封装成对象或用构建器模式等优化。返回值类型明确，避免用模糊类型（如 Object）。
13. 代码块（if、else、for、while、switch 等）中用大括号 {} 包围代码体，即使只有一行。
14. 捕获异常时尽量捕获具体异常类型，不笼统捕获 Exception 或 Throwable。
15. 方法可能抛出受检异常时，用 throws 关键字明确声明抛出异常类型。
16. 多线程环境下确保共享资源访问线程安全，通过同步机制（synchronized、ReentrantLock 等）、不可变对象、线程安全集合类等方式保证。
17. 使用线程池时合理设置参数（核心线程数、最大线程数、队列容量、拒绝策略等），避免资源耗尽或任务积压。
18. 引入依赖时确保版本号明确，不用模糊版本号（如 LATEST、RELEASE）。优先用稳定版本依赖，避免预发布或不稳定版本，除非充分测试验证。
19. 定期检查项目依赖更新，特别是安全漏洞修复和重要功能改进版本。更新依赖时全面测试，确保新版本不导致功能异常和兼容性问题。
20. 每个类和方法都有相应注释，解释功能、用途和主要逻辑。类注释包括整体描述、关键属性及方法说明；方法注释包括功能描述、参数、返回值及异常说明。
21. 复杂逻辑块、算法实现或关键业务逻辑代码添加详细注释，解释设计思路、实现原理和关键点。注释紧跟代码块，清晰描述逻辑流程和特殊处理。

22. 合理使用日志级别 (DEBUG、INFO、WARN、ERROR)，依场景和重要性记录日志。DEBUG 记录详细调试信息，生产环境少用；INFO 记录正常运行重要信息；WARN 记录可继续运行潜在错误；ERROR 记录严重错误异常。
23. 避免日志记录敏感信息 (如密码、用户隐私数据)，除非严格安全措施保护。记录异常信息时保证异常堆栈信息完整。
24. 获取资源 (文件、数据库连接、网络连接等) 时，确保使用完毕正确释放，防止资源泄漏。用 try - finally 或 try - with - resources 等结构确保释放。
25. 合理设置资源获取使用方式，避免过度获取浪费。如合理设置数据库连接池、文件读写缓冲区大小等，提高效率性能。
26. 代码提交版本控制系统前须经代码审查。至少一名其他开发人员审查，内容包括规范性、逻辑正确性、安全性、可读性等。只有通过审查代码才能合并主分支。
27. 代码审查重点关注：是否符合规范文档规则；有无潜在逻辑错误或安全隐患；是否影响现有功能；是否添加必要测试用例。审查人员需认真负责，提出具体修改建议。

二、推荐

1. 在代码块中保持语句对齐，增强可读性。较长代码块可用括号等辅助对齐。
2. 避免过长代码行，提前逻辑拆分和封装，使代码简洁易读。
3. 在代码中适当添加空格，增强复杂表达式和语句的可读性。
4. 在较长方法内部根据逻辑功能适当添加空行，使代码更清晰。
5. 局部变量可适当简化命名，类成员变量和全局变量使用更详细命名，避免命名冲突和歧义。
6. 类名和接口名简洁明了，避免过长和缩写 (除非团队广泛认可且无歧义)。
7. 方法名简洁明了，避免过长和缩写 (除非团队广泛认可且无歧义)，清晰传达输入、输出和副作用等信息。
8. 方法长度控制在 20 行内，最长不超 40 行。过长方法拆分为多个小方法。方法内嵌套层次不超 3 层，过多时提取独立方法。
9. 使用标准注释格式 (如 Javadoc 格式)，便于生成文档查阅。注释简洁明了，避免冗长重复，突出核心要点。
10. 注释中使用示例、图示或伪代码辅助说明复杂逻辑，提高可读性和理解性。
11. 记录日志时提供足够上下文信息 (如方法名、类名、时间戳、用户信息等)，便于定位问题。
12. 使用日志框架功能 (如 MDC) 记录当前线程相关上下文信息，方便多线程环境跟踪分析日志。
13. 使用资源管理框架或工具类统一管理资源获取释放，降低泄漏风险。
14. 定期监控资源使用情况 (文件描述符、数据库连接数、内存等)，及时发现优化瓶颈。
15. 建立代码审查流程规范，明确步骤、标准和工具。可用 Gerrit、CodeReview、Pull Request 等工具辅助审查。

16. 审查除基本规范和逻辑外，关注代码可维护性和可扩展性，鼓励编写高质量可复用代码。

三、允许

1. 在特殊情况下（如注释格式化）适当调整缩进方式，但尽量保持一致性。
2. 无法拆分的字符串或注释适当放宽行长度限制，但尽量避免。
3. 简短表达式或语句适当减少空格使用，但保持一致性。
4. 方法体简短时适当减少空行数量，避免过多空行导致代码分散。
5. 特殊情况（如特定设计模式或性能优化）调整类结构顺序，但保证可读性和可维护性。
6. 算法实现或复杂业务逻辑等特殊情况方法长度和嵌套层次超推荐值，但需注释解释。
7. 简化代码或快速原型开发等特殊情况暂时捕获 `Exception` 或 `Throwable`，但尽快替换为具体异常处理方式。
8. 特定错误处理机制或兼容第三方库等特殊情况适当抛出运行时异常，但保证不严重影响程序稳定性。
9. 性能要求极高单线程场景适当放宽线程安全要求，但确保无多线程访问。
10. 测试环境或临时任务等特殊情况使用默认线程池参数，但保证不严重影响系统性能和稳定性。
11. 为用特定功能或修复 bug 等特殊情况临时引入预发布或不稳定版本依赖，但尽快升级替换。
12. 项目紧急发布等特殊情况适当推迟依赖更新，但尽快安排更新测试。
13. 简单 `getter` 和 `setter` 方法适当简化注释，但不引起歧义。
14. 通用、简单代码逻辑适当减少注释，但代码本身具良好可读性。
15. 特殊情况（如统计或监控特定业务指标）自定义日志级别或格式，但与整体日志管理系统兼容。
16. 根据项目需求和性能要求，调整日志输出和保留策略。
17. 资源自动回收可靠或使用频率极低等特殊情况适当简化资源释放逻辑，但不影响系统性能稳定性。
18. 应对临时高并发等特殊情况调整资源获取使用策略，但尽快恢复。
19. 紧急情况（如快速修复线上严重 bug）暂时跳过代码审查，事后尽快补流程，保证代码质量。
20. 根据项目特点和需求调整代码审查重点范围，但不遗漏关键问题。